UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Câmpus de Presidente Prudente

UNESP - Faculdade de Ciências e Tecnologia
Bacharelado em Ciência da Computação

Gabrielly Christina Moreira
Marcos Andre Trajano da Silva Junior
Yasmin de Oliveira Bonet

TRABALHO II
Q-learning

Disciplina: Inteligência Artificial
Docente: Prof. Danillo Roberto Pereira

Presidente Prudente-SP
2026

UNESP - Faculdade de Ciências e Tecnologia
Bacharelado em Ciência da Computação

Gabrielly Christina Moreira
Marcos Andre Trajano da Silva Junior
Yasmin de Oliveira Bonet

TRABALHO II
Q-learning

Trabalho apresentado à disciplina de Inteligência Artificial, do curso de Bacharelado em Ciência da Computação, como requisito parcial para avaliação do Trabalho 2 - Q-learning.
Prof.: Danillo Roberto Pereira

Presidente Prudente-SP
2026

Sumário:

1. Introdução: 4
2. Jogo: 4
2.1 Ações do jogo: 4
2.2 Chute, Recompensas e Fim da Partida: 5
2.3 Goleiro: 5
2.4 Estado: 6
3. Q-learning aproximado: 6
3.1 Parâmetros do Algoritmo: 6
3.2 Features: 7
3.3 Conjuntos de Features Comparados: 7
3.4 Treino e Avaliação: 7
4. Implementação: 8
4.1 Como executar: 8
4.2 Treino com Parâmetros Padrões: 8
5. Experimentos e resultados: 10
5.1 Alpha (taxa de aprendizado): 10
5.2 Gamma (desconto do futuro): 11
5.3 Epsilon (Exploração): 12
5.4 Números de Episódios: 13
5.5 Conjuntos de Features: 13
5.6 Comportamento do goleiro: 14
6. Discussão: 15
7. Conclusão: 16

1. Introdução:

Este presente trabalho implementa um jogo de cobrança de gol e um agente que aprende a jogá-lo com Q-learning aproximado, e mostra como os parâmetros do algoritmo mudam o aprendizado. Tudo foi feito em Python, usando somente as bibliotecas numpy, pandas e matplotlib, como pede o enunciado.

O Q-learning é um algoritmo de aprendizado por reforço: o agente joga muitas partidas, recebe recompensas (positivas ou negativas) e aos poucos descobre quais ações valem a pena em cada situação. Na versão tabular, ele guarda um valor para cada par situação–ação. Na versão aproximada, ele descreve cada situação por algumas características numéricas (as features) e aprende um peso para cada uma, o que dispensa a tabela.

No jogo criado, o jogador atravessa o campo até uma área de chute e precisa chutar quando o goleiro não está na sua linha. Com os parâmetros padrão, o agente treinado marcou gol em 100% das partidas de avaliação, contra cerca de 1,7% de um agente que joga ao acaso. O agente que não enxerga o goleiro marcou 0%, o que mostra o peso da escolha das features.

2. Jogo:

O jogo é uma cobrança de gol simplificada: o jogador precisa chegar à área de chute e chutar quando o goleiro não está na mesma linha que ele. O campo é uma grade de 5 linhas por 7 colunas. O jogador começa na coluna 0, na linha do meio, enquanto o goleiro fica na última coluna (a do gol), em uma linha sorteada no início de cada partida.

A área de chute são as colunas 4 e 5. O jogador não pode entrar na coluna do gol, então a coluna 5 é o ponto mais avançado que ele alcança.

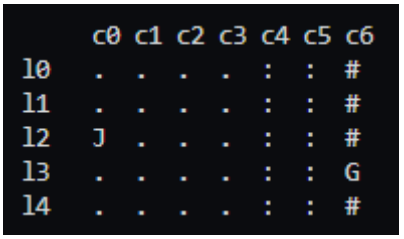


Imagem 1: O campo desenhado pelo jogo.py.

Esse é como o campo é desenhado para o jogo, sendo cada elemento:

- **l0, ..., l4:** Linhas do campo;
- **c0, …, c6:** Colunas do campo;
- **J:** Jogador
- **G**: Goleiro
- **#:** Gol;
- **: (dois pontos):** Área de chute.

2.1 Ações do jogo:

A cada turno o jogador escolhe uma entre as quatro acções possíveis:

Ação:	O que faz:
Subir	Vai uma linha para cima (sem sair do campo).
Descer	Vai uma linha para baixo (sem sair do campo).
Avançar	Vai uma coluna para a frente (até a coluna 5) .
Chutar	Chuta em linha reta pela linha onde está e encerra a partida .

Tabela 1: Ações do jogador.

2.2 Chute, Recompensas e Fim da Partida:

O chute é resolvido na hora, antes do turno e o goleiro se mexer. A partida é encerrada ao realizar o gol, ter defesa defesa do goleiro, ter chute fora da área ou ao fim de 40 turnos, sendo cada turno que passa custando -1 ponto, o que incentiva o agente a ser rápido.

Lance:	Situação:	Recompensa:	Partida:
Chutou dentro da área e o goleiro não está na linha.	Gol.	+100 pontos	Encerra.
Chutou dentro da área e o goleiro está na linha.	Defesa.	-50 pontos	Encerra.
Chutou antes de chegar à área.	Fora.	-20 pontos	Encerra.
Qualquer outro turno.	Próxima rodada.	-1 ponto	Continua para o próximo turno (máx. 40).

Tabela 2: Lances e situações do jogo.

2.3 Goleiro:

Após cada ação do jogador que não seja o chute a gol, o goleiro reage e o comportamento dele é um parâmetro do jogo e tem três modos:

- **Reativo (padrão):** a cada período de turnos, o goleiro dá um passo em direção à linha do jogador. O período padrão é 2;
- **Sorteio:** a cada turno, o goleiro tem 50% de chance de dar um passo em direção à linha do jogador.
- **Patrulha:** o goleiro sobe e desce sozinho, sem olhar o jogador.

O período do modo reativo controla a dificuldade. Com o período 1, o goleiro acompanha o jogador a cada turno e o jogo fica impossível de ganhar. Isso foi confirmado por uma busca exaustiva, que não encontrou nenhuma sequência vencedora em 100 posições iniciais testadas. Com período 2 ou mais, existe sempre um jeito de ganhar.

2.4 Estado:

O estado do jogo guarda: a linha e a coluna do jogador, a linha do goleiro, se o jogador está na área, se o goleiro vai se mexer logo após a próxima ação (no modo reativo) e também o número do turno. O agente não usa esse estado diretamente: ele o transforma em features, que será mais desenvolvido .

3. Q-learning aproximado:

O agente aprende uma função $Q(s, a)$ que estima o quanto vale fazer a ação a na situação s, e escolhe a ação de maior valor. Na versão aproximada, essa função é uma soma ponderada de features, em vez de uma tabela com todas as situações:

$$Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \ldots + w_n f_n(s,a)$$

Os pesos w começam em zero e são ajustados a cada jogada. Depois de executar a ação a na situação s, receber a recompensa r e chegar à situação s', o agente calcula o erro de diferença temporal e corrige cada peso:

$$erro = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

$$w_i \leftarrow w_i + \alpha \,.\, erro \,.\, f_i(s,a)$$

Quando a partida termina em s', o termo do futuro (gamma vezes o melhor Q seguinte) vale zero, e o alvo é apenas a recompensa r.

```python
def atualizar(self, features, recompensa, jogo_novo, terminou):
    q_atual = float(self.pesos @ features)
    if terminou:
        alvo = recompensa
    else:
        alvo = recompensa + self.gamma * self.melhor_valor(jogo_novo)
    erro = alvo - q_atual
    self.pesos += self.alpha * erro * features
```

Imagem 2: Método atualizar da classe AgenteQLearning, que aplica as fórmulas citadas.

3.1 Parâmetros do Algoritmo:

Parâmetro:	O que controla:	Valor padrão:
Alpha	Taxa de aprendizado: o quanto cada erro ajusta os pesos.	0,5

Gamma	O quanto o agente valoriza recompensas futuras (0 a 1).	0,9
Epsilon	Chance de escolher uma ação ao acaso (exploração).	0,1
Episódios	Quantas partidas o agente joga aprendendo.	3000

Tabela 3: Parâmetros e o que controlam.

3.2 Features:

Para cada ação possível, o agente calcula 8 features da situação que resultaria dela, todas entre 0 e 1, sendo elas:

- **`bias`:** Sempre 1. Dá ao agente um valor de partida;
- **`falta_para_area`:** Quanto falta para chegar à área depois da ação (0 se já está na área, 1 se está na coluna 0);
- **`chute_fora`:** 1 se a ação pe chutar antes de chegar à área;
- **`chute_na_area`:** 1 se a ação é chutar dentro da área;
- **`chute_gol`:** 1 se a ação é chutar dentro da área e o goleiro não está na linha do jogador;
- **`chute_defendido`:** 1 se a ação é chutar dentro da área e o goleiro está na linha do jogador;
- **`alinhado_depois`:** 1 se, depois da ação e da reação do goleiro, ele ficará na linha do jogador (só para ações de movimento);
- **`dist_goleiro_depois`:** Distância vertical até o goleiro depois da ação e da reação dele, dividida por 4 (somente para ações de movimentos).

Sendo que as features `alinhado_depois` e `dist_goleiro_depois` usam a informação de se o goleiro vai se mexer neste turno ou não, permitindo ao agente aprender o "momento certo", se o goleiro está alinhado, mas só vai se mexer no turno seguinte ou se vale a pena sair da linha agora e chutar depois.

3.3 Conjuntos de Features Comparados:

O código permite treinar com dois conjuntos de features, o conjunto completo usa as 8 features totais, o de sem goleiro usa só `bias`, `falta_para_area`, `chute_fora` e `chute_na_area` , então o agente não sabe onde o goleiro está.

3.4 Treino e Avaliação:

No treino, o agente joga com exploração e atualiza os pesos a cada jogada, enquanto na avaliação,ele joga sem explorar e sem aprender, em partidas novas. Por isso, a taxa de gols no treino (que inclui as jogadas aleatórias da exploração) costuma ser menor que a taxa de gols vista na avaliação.

4. Implementação:

O trabalho completo está dividido em três arquivos de linguagem Python, que ficam localizados na mesma pasta e utilizam somente as bibliotecas numpy, matplotlib e pandas. Os códigos são identificados como:

- **`jogo.py:`** Contém as regras do jogo, presente na classe `JogoFutebol`, o desenho do campo no terminal e dois agentes de referência: o aleatório e o de regras;
- **`agente.py:`** Contém as features (`calcular_features`), a classe `AgenteQLearning` e as funções treinar e avaliar;
- **`experimentos.py:`** Contém os 6 experimentos totais realizados, treina o agente variando um parâmetro por vez, repete com 5 sementes e salva as tabelas em .csv e os gráficos em .png na pasta 'resultados'.

Os dois agentes de referência servem de piso e de teto para comparar com o agente que aprende. O agente aleatório escolhe qualquer ação em cada turno e marca gol em cerca de 1,7% das partidas. O agente de regras foi escrito à mão, sem aprendizado: avança até a área, chuta quando o goleiro está fora da linha e, se estiver alinhado, se afasta dele, marcando gol em 100% das partidas, o que mostra que o jogo pode ser vencido.

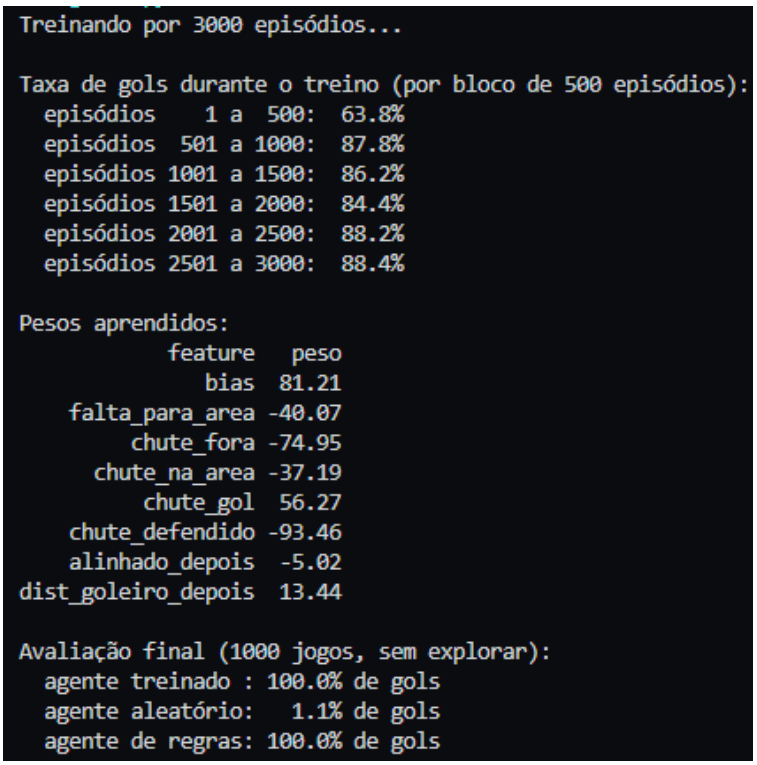
4.1 Como executar:

Para executar os códigos, basta:

1. Verificar se os três arquivos (`jogo.py`, `agente.py` e `experimentos.py`) estão localizados na mesma pasta;
2. Rodar `jogo.py`, que mostra um jogo de exemplo e a taxa de gols dos agentes de referência em cada modo do goleiro;
3. Rodar `agente.py`, que treina o agente por 3000 episódios com os parâmetros padrão e faz a avaliação final;
4. Rodar `experimentos.py`, em que executa os 6 experimentos e cria a pasta 'resultados'.

4.2 Treino com Parâmetros Padrões:

No treino com alpha 0,05, gamma 0,9, epsilon 0,1 e 3000 episódios, a taxa de gols durante o treino subiu de 63,8% (episódios 1 a 500) para cerca de 88% (episódios 2501 a 3000). Na avaliação final, sem exploração, o agente treinado marcou gol em 100% das 1000 partidas, contra 1,1% do agente aleatório e 100% do agente de regras.



```
Treinando por 3000 episódios...

Taxa de gols durante o treino (por bloco de 500 episódios):
  episódios    1 a  500:  63.8%
  episódios  501 a 1000:  87.8%
  episódios 1001 a 1500:  86.2%
  episódios 1501 a 2000:  84.4%
  episódios 2001 a 2500:  88.2%
  episódios 2501 a 3000:  88.4%

Pesos aprendidos:
            feature   peso
               bias  81.21
    falta_para_area -40.07
        chute_fora -74.95
      chute_na_area -37.19
         chute_gol  56.27
    chute_defendido -93.46
    alinhado_depois  -5.02
dist_goleiro_depois  13.44

Avaliação final (1000 jogos, sem explorar):
  agente treinado : 100.0% de gols
  agente aleatório:   1.1% de gols
  agente de regras: 100.0% de gols
```

Imagem 3: Saída completa do agente.py.

Os pesos aprendidos dessa execução mostram o que cada agente descobriu:

Feature:	Peso:	Leitura:
`bias`	81,21	Valor de partida.
`falta_para_area`	-40,07	Avançar até a área é bom.
`chute_fora`	-74,95	Chutar antes da área é ruim.
`chute_na_area`	-37,19	Compensa o `bias` ao chutar.
`chute_gol`	56,2	Chutar com o goleiro fora da linha é muito bom.
`chute_defendido`	-93,46	Chutar com o goleiro na linha é muito ruim.
`alinhado_depois`	-5,02	Ficar na linha do goleiro é ligeiramente ruim.
`dist_goleiro_depois`	13,44	Ficar longe do goleiro é bom.

Tabela 4: Significado dos pesos na leitura das features.

Um teste simples de que o aprendizado está coerente: o Q de um chute que termina em gol é 81,21 - 37,19 + 56,27 = 100,29, muito próximo da recompensa real de +100. O Q de um chute defendido é 81,21 - 37,19 - 93,46 = -49,44, próximo da recompensa real de -50.

5. Experimentos e resultados:

Ao todo, foram realizados 6 diferentes experimentos, cada um variando algum parâmetro e mantendo outros no valor de padrão, como: alpha 0,05; gamma 0,9; epsilon 0,1; 3000 episódios; features completas; goleiro reativo com período 2.

- Cada configuração foi treinada 5 vezes com sementes diferentes, e as tabelas mostram a média.
- **Gols na avaliação:** taxa de gols em 500 partidas novas, jogadas sem explorar e sem aprender. O desvio é o desvio padrão entre as 5 sementes.
- **Gols no treino:** taxa de gols nos últimos 500 episódios de treino, que inclui as jogadas aleatórias da exploração.
- **Episódios até 80%:** primeiro episódio em que a curva média de gols no treino passa de 80%. Como a curva usa média móvel de 100 episódios, o menor valor possível é cerca de 100.
- Como referência, o agente aleatório marca gol em cerca de 1,7% das partidas.
- Em cada gráfico, o painel da esquerda mostra a curva de aprendizado e o da direita mostra o resultado final de cada configuração, com a linha tracejada do agente aleatório.

5.1 Alpha (taxa de aprendizado):

Um alpha muito pequeno atrasa bastante o aprendizado, mas não muda o resultado final com 3000 episódios.

Alpha:	Gols na avaliação (%):	Gols no treino:	Episódios até 80%:
0,001	100	84,9	1444
0,0,	100	86,8	242
0,05	100	87,2	197
0,1	100	86,7	109
0,2	100	86,3	169

Tabela 5: Resultados alpha.

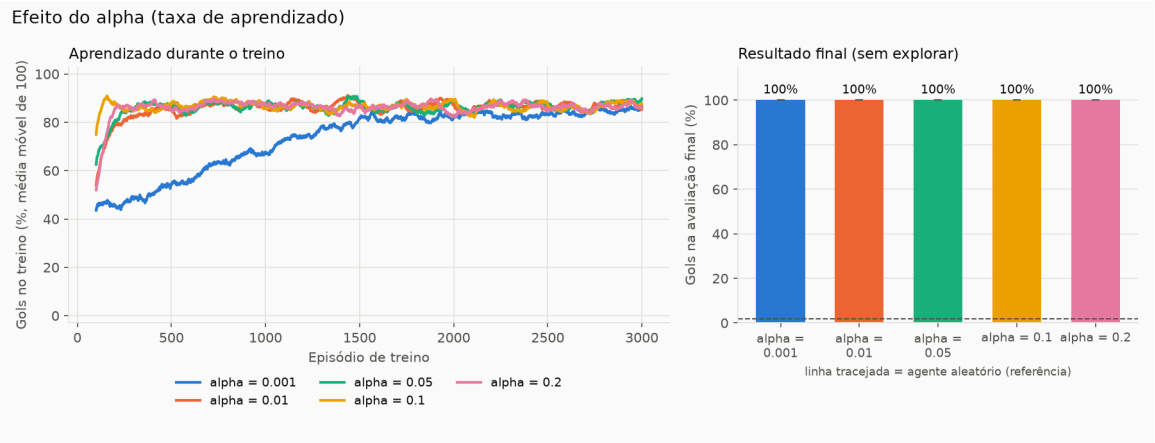


Imagem 4: Resultados alpha.

Com alpha 0,001, o agente precisa de 1444 episódios para passar de 80% de gols no treino, enquanto com 0,01 a 0,2 precisa de 109 a 242. Isso acontece porque, com alpha pequeno, cada ajuste dos pesos é minúsculo. Entre 0,05 e 0,2 as diferenças são pequenas, e com 5 sementes não dá para afirmar qual valor é o melhor.

5.2 Gamma (desconto do futuro):

Com gamma 0 o agente não aprende, e a partir de 0,5 ele aprende, mais rápido quanto maior o gamma.

Gamma:	Gols na avaliação (%):	Gols no treino (%):	Episódios até 80%:
0	0	30,2	Não atingiu.
0,5	100	86,7	314
0,9	100	87,2	197
0,99	100	86,4	105

Tabela 6: Resultados gamma.



Imagem 5: Resultados gamma.

Com gamma 0, o agente só considera a recompensa imediata. A explicação provável é que a recompensa de gol só aparece no chute, então avançar até a área parece apenas custar -1 por turno, e o agente não aprende a chegar lá. Com gamma 0,99 foram necessários 105 episódios para passar de 80% no treino, contra 314 com gamma 0,5.

5.3 Epsilon (Exploração):

Sem exploração (epsilon 0) o agente não aprende, e com epsilon alto ele aprende, mas joga pior durante o treino.

Epsilon:	Gols na avaliação (%):	Gols no treino (%):	Episódios até 80%:
0	0	0,0	Não atingiu.
0,05	100	94,5	745
0,1	100	87,2	197
0,3	100	60,1	Não atingiu.
0,6	100	25,2	Não atingiu.

Tabela 7: Resultados epsilon.



Imagem 6: Resultados epsilon.

Com epsilon 0 o agente nunca marcou gol, nem no treino nem na avaliação. A explicação provável é que, sem tentar ações diferentes das que já considera melhores, ele nunca experimenta chutar no momento certo. De 0,05 a 0,6 o agente chega a 100% na avaliação, mas a taxa de gols no treino cai de 94,5% para 25,2%, porque a maior parte das jogadas passa a ser aleatória. O epsilon 0,05 chegou a mais gols no treino, porém demorou mais para passar de 80% (745 episódios, contra 197 com epsilon 0,1).

5.4 Números de Episódios:

Com 300 episódios o agente já chega a 100% em todas as sementes, enquanto com 50 ou 100 o resultado é instável.

Episódios:	Gols na avaliação (%):	Desvio (%):	Gols no treino (%):
50	80	40	54,8
100	80	40	62,6
300	100	0	76,2
1000	100	0	86,9
3000	100	0	87,2

Tabela 8: Resultados dos episódios.

Para 50, 100 e 300 episódios, a taxa de gols no treino considera todos os episódios do treino, e não só os últimos 500.

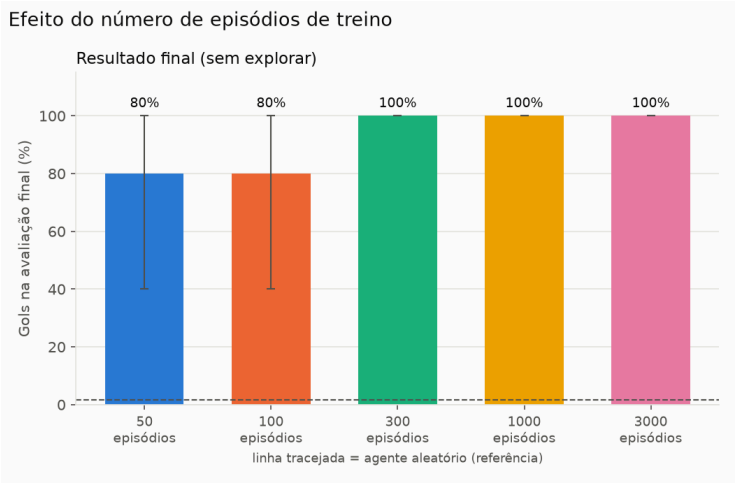


Imagem 7: Resultados dos episódios.

A média de 80% com desvio de 40 pontos é consistente com 4 sementes chegando a 100% e 1 semente ficando em 0%: com poucos episódios, o agente às vezes ainda não aprendeu. Depois de 300 episódios, mais treino quase não muda o resultado: de 1000 para 3000 episódios a taxa de gols no treino sobe apenas de 86,9% para 87,2%.

5.5 Conjuntos de Features:

Sem enxergar o goleiro, o agente não aprende a marcar gol, e essa é a maior diferença de todos os experimentos.

Features:	Gols na avaliação (%):	Gols no treino (%):	Episódios até 80%:

Completo (8 features)	100	87,2	197
Sem goleiro (4 features)	0	14	Não atingiu.

Tabela 8: Resultados das features.

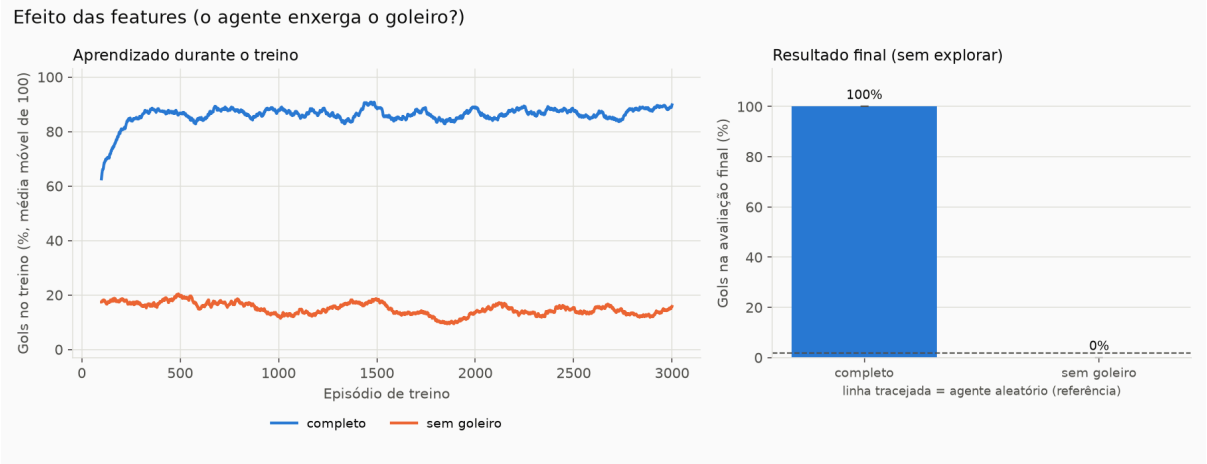


Imagem 8: Resultados dos features.

Sem as features do goleiro, o agente só sabe se está dentro ou fora da área. Isso importa porque, no modo reativo com período 2, o goleiro já está alinhado ao jogador em 100% dos jogos quando ele chega à área em linha reta (teste com 2000 jogos). Chutar sem olhar o goleiro termina em defesa (-50), e o agente aprende que chutar vale menos que as outras ações: em um treino com semente 1, o Q de chutar na área ficou em -11,2, contra -8,2 das demais ações. Com as mesmas regras, o mesmo algoritmo e os mesmos parâmetros, só a escolha das features separa 100% de 0%.

5.6 Comportamento do goleiro:

O agente aprende em todos os modos do goleiro em que o jogo pode ser vencido, e fica em 0% com período 1, em que o jogo é impossível.

Goleiro:	Gols na avaliação (%):	Gols no treino (%):	Episódios até 80%:	Agente relatório (%):
Reativo, período 1	0	0,0	Não atingiu.	0,0
Reativo, período 2	100	87,2	197	1,7
Reativo, período 3	100	87,4	100	2,4
Reativo,	100	87,9	100	3,0

período 4				
Sorteio 50%	100	86,5	103	1,8
Patrulha	100	89,7	100	4,8

Tabela 9: Resultados dos comportamentos dos goleiros.

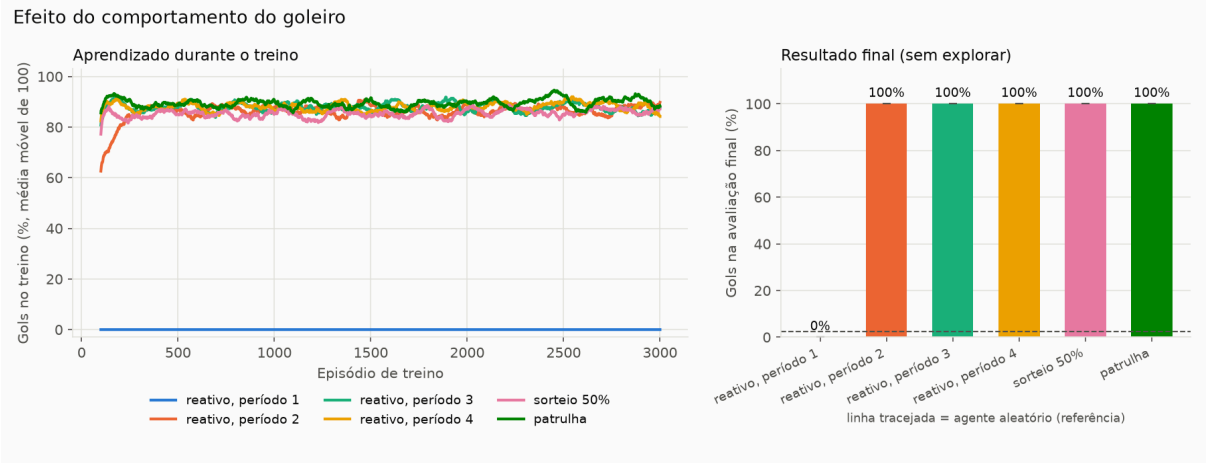


Imagem 8: Resultados dos comportamentos dos goleiros.

Quanto mais lento o goleiro, mais fácil é o jogo: o agente aleatório sobe de 1,7% (período 2) para 3,0% (período 4), e o agente treinado passa de 80% no treino em cerca de 100 episódios, contra 197 com período 2. No modo patrulha, em que o goleiro ignora o jogador, o agente aleatório chega a 4,8% e o agente treinado também atinge 100%. No modo sorteio o agente também chega a 100%, mesmo com features pensadas para o modo reativo.

6. Discussão:

Neste jogo, a escolha das features pesou mais do que o ajuste fino dos parâmetros. Os principais pontos dos experimentos são:

1. **Features:** Com o mesmo algoritmo e os mesmos parâmetros, o agente com todas as features chegou a 100% e o agente sem as features do goleiro ficou em 0% (experimento 5). O agente só aprende aquilo que as features conseguem representar, e aqui a posição do goleiro e o momento em que ele se mexe são essenciais.
2. **Exploração e visão de futuro:** Com epsilon 0 e com gamma 0 o agente não aprendeu (experimentos 2 e 3). Com valores intermediários ele aprende, e mais exploração reduz os gols durante o treino sem prejudicar a avaliação final.
3. **Velocidade, não resultado final:** Alpha, gamma e o número de episódios mudaram principalmente a velocidade do aprendizado: alpha 0,001 precisou de 1444 episódios para passar de 80% no treino, e 300 episódios bastaram com os parâmetros padrão.
4. **Dificuldade do jogo:** Com o goleiro acompanhando o jogador a cada turno (período 1) o jogo é impossível, e goleiros mais lentos deixam o jogo mais fácil tanto para o agente aleatório quanto para o agente treinado.

5. **Estratégia aprendida:** O agente reproduziu a estratégia do agente de regras (avançar até a área e chutar quando o goleiro está fora da linha), e os pesos aprendidos confirmam isso.

7. Conclusão:

Com o trabalho cumprindo as orientações propostas: um jogo implementado do zero, com algoritmo de Q-learning aproximado e a análise do efeito de diferentes parâmetros, usando somente numpy, pandas e matplotlib. O agente de Q-learning aproximado aprendeu a jogar o jogo do goleiro e marcou gol em 100% das partidas de avaliação, contra cerca de 1,7% do agente aleatório. A escolha das features foi o fator decisivo (100% contra 0%), seguida da exploração e do desconto do futuro, sem os quais o agente não aprende.

Alpha, gamma e o número de episódios alteraram principalmente a velocidade do aprendizado. Com alpha 0,001 foram necessários 1444 episódios para passar de 80% de gols no treino, e 300 episódios foram suficientes com os parâmetros padrão.